RAG Assignment: Baseline vs. Re-Ranking

Bi-Encoder Retrieval vs. Cross-Encoder Re-Ranking on the FIDE Laws of Chess

Kushal Mukherjee
Roll No.: 2025201072

19 October 2026

Code: `https://github.com/Kushal-2002/rag_Assignment` (single notebook: `rag_reranking.ipynb`)

1 Setup

Component	Choice
Document	FIDE Laws of Chess (25 pages: Articles 1–14, Appendices A–F, Guidelines)
Chunking	One chunk per numbered clause (3.8, 9.2, A.4, ...), max. 150 words → 132 chunks
Bi-Encoder	`sentence-transformers/all-MiniLM-L6-v2` (384-dim)
Vector store	FAISS `IndexFlatIP` on normalised vectors (= cosine similarity)
Cross-Encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`
LLM	`llama3` (8B) via local Ollama, temperature 0

Both pipelines use the same prompt, which tells the LLM to answer *only* from the three given chunks and otherwise reply "Not found in the context." This stops the model from answering from memory and hiding a retrieval failure. Without any context, llama3 already stated a rule wrongly (it said the king moves *three* squares when castling; the correct number is two).

2 Architecture Comparison

Bi-Encoder. The query and each chunk are encoded *independently* by the same transformer, and each is mean-pooled into a single vector. Relevance is the cosine similarity of the two vectors:

$$s_{\text{bi}}(q, d) = \cos\big(E(q),\, E(d)\big).$$

Because the chunk vectors do not depend on the query, they can be computed once, offline, and stored in a vector index. A query then needs only one encoding plus a fast nearest-neighbour search. The drawback is that the query and chunk tokens never interact: the whole chunk is compressed into one 384-dimensional vector *before* the model has seen the question. A small detail such as "in rapidplay", "for a blind player" or "not" therefore has little effect on the score.

Cross-Encoder. The query and chunk are concatenated into one input, `[CLS] query [SEP] chunk [SEP]`, and passed through the transformer together. Self-attention runs *jointly* across all tokens, so every query word can attend to every chunk word. A classification head outputs one relevance score:

$$s_{\text{ce}}(q, d) = f\big([\,q\,;\,d\,]\big).$$

This is much more precise, but nothing can be pre-computed: every (query, chunk) pair needs its own full forward pass at query time.

3 Qualitative & Rank Shift Analysis

3.1 Method

About 160 candidate queries were written across the four failure types. Each query was labelled *manually* with the chunk(s) that actually answer it, found by reading the document. For every query, the rank of the correct chunk was recorded before and after re-ranking. A query counts as a failure case when the correct chunk is **outside the baseline Top 3** but **inside the Top 3 after re-ranking**. Of the 30 representative queries kept in the notebook, **10 meet this criterion**.

3.2 Rank shifts

Type	Query	Chunk	Bi → CE rank
Document-specific	In rapidplay, after how many moves can you no longer complain about piece placement?	A.4	#7 → #1
Keyword trap	In blind chess, how is castling announced?	E.2	#6 → #1
Keyword trap	What symbol is used to mark a check?	C.13	#5 → #1
Document-specific	For a blind player, when is a piece considered touched?	E.2 (pt. 2)	#4 → #1
Exception	What happens if the starting position of the pieces was wrong in rapidplay?	A.4	#7 → #2
Negation	Can a player not move a piece again once he released it?	4.6	#6 → #2
Exception	In Chess960, does the king always move when castling?	F.3 (Notes)	#5 → #2
Document-specific	In rapidplay, does the arbiter rule on touch-move on his own?	A.4	#5 → #2
Keyword trap	What names are used for the files in blind chess?	E.2	#8 → #3
Exception	Do the special touch rules for blind players also apply to the sighted player?	E.2 (pt. 2)	#7 → #3

Why the baseline fails here. Nearly all failures follow one document-specific pattern. The Laws state a rule in the main Articles and then restate a *variant* of it in an appendix (Rapidplay, Blind players, Chess960). For "In blind chess, how is castling announced?", the bi-encoder is drawn to the word *castling* and fills its Top 3 with the standard castling rule (3.8) and the Chess960 castling chunks. The cross-encoder reads the whole question against each chunk, sees that it asks about *announcing* a move in *blind* chess, and promotes Appendix E ("Lange/Kurze Rochade") to #1. The negation case is similar: for "Can a player *not* move a piece again once he released it?", the bi-encoder ranks 4.5 (touched pieces) and 7.3 (displaced pieces) higher because they share the words *piece* and *move*. The baseline Top 3 did contain the *second half* of 4.6, but not the sentence with the restriction. The cross-encoder promotes the first half of 4.6, which states it ("it cannot be moved to another square on this move").

3.3 Answer 1 (baseline) vs. Answer 2 (re-ranked)

Query	Answer 1 (baseline Top 3)	Answer 2 (re-ranked Top 3)
Rapidplay: complain about piece placement	Not found in the context.	After three moves, no claim can be made regarding incorrect piece placement, board orientation or clock setting. **(correct)**
Blind chess: castling announced	Not found in the context.	Announced as "Lange Rochade" (long) and "Kurze Rochade" (short). **(correct)**
Blind player: piece touched	Not found in the context.	When it has been taken out of the securing aperture. **(correct)**
Released piece (negation)	Not found in the context.	Yes, once released on a square it cannot be moved to another square on this move. **(correct)**
Chess960: king always moves?	Not found in the context.	Not found in the context. **(chunk at #2, LLM missed it)**
Rapidplay: arbiter and touch-move	Not found in the context.	Not found in the context. **(chunk at #2, LLM missed it)**

Observations.

- **Omission rather than hallucination.** Because of the strict prompt, the baseline usually did not invent rules. It *omitted* the answer ("Not found"), even though the correct clause was sitting at rank 4–7. Re-ranking turned these omissions into correct, clause-grounded answers.
- **Misstated rule when the context is wrong.** For "When is castling not allowed?", Answer 1 listed the temporary restrictions and then ended with "So, the answer is: Not found in the context.", contradicting itself because two of its three chunks were weak. For "How is a win on time claimed in rapidplay?", the correct chunk was not in the Top 10 at all. Answer 2 then confidently described the Appendix D *draw* claim as the way to claim a *win*, which is a misstated rule.
- **Better ranking $\neq$ better answer.** In two cases the correct chunk reached #2, but llama3 (8B) still replied "Not found". Retrieval quality is necessary but not sufficient; the generator's reading ability also matters.
- **Re-ranking can also hurt.** In 3 of the 30 queries the cross-encoder pushed the correct chunk *out* of the Top 3 (e.g. "Can I go to the bathroom during my game?": #1 $\rightarrow$ #4). The MS MARCO model was trained on web search, not on legal-style rule text.
- **The bi-encoder is strong on this document.** Most queries that reuse the rulebook's own wording were already answered correctly by the baseline. Re-ranking mainly helps on paraphrased questions and rule variants.

4 Latency Profiling

Times were measured with `time.perf_counter()` on a laptop CPU (Apple Silicon) and averaged over the 7 selected queries.

Stage	Baseline (ms)	Re-ranking (ms)
Bi-encoder retrieval (Top 10)	111	111
Cross-encoder re-ranking (10 pairs)	–	149
LLM generation (llama3, 8B)	2,861	3,615
Total	**2,973**	**3,876**

- Re-ranking adds about **150 ms per query** (about 15 ms per pair). This roughly doubles the retrieval stage, but it is only about 5% of the end-to-end time.
- The LLM accounts for more than 90% of the total. The re-ranking pipeline's LLM time is higher mainly because it produced real (longer) answers, whereas the baseline often replied with the short "Not found" sentence. It is not a cost of re-ranking itself.
- Single runs vary considerably on a CPU (retrieval ranged from 6 to 264 ms), so averages are reported.

5 Scaling Question

Why not run the Cross-Encoder over the whole collection from the start?

- **Nothing can be pre-computed.** A bi-encoder embeds all documents once, offline. At query time it encodes only the query and runs a nearest-neighbour search, which is sub-linear with approximate indexes such as HNSW or IVF. A cross-encoder needs the query *inside* its input, so every document must be re-processed for every new query.
- **Cost grows linearly with the collection.** With our measured ≈15 ms per pair, scoring all 132 chunks would take about 2 s per query, already slower than the LLM's share of the work. For 1 million passages it would be $10^6 \times 15\,\text{ms} \approx 4$ hours *per query*, compared with milliseconds for a vector search.
- **Retrieve-then-re-rank gets the best of both.** The cheap bi-encoder narrows millions of documents to a small candidate set (here 10) with high recall. The expensive cross-encoder is then spent only where precision matters, on ordering those few candidates.
- **Trade-off.** The cross-encoder can only re-order what the bi-encoder returns. When the correct chunk is not in the Top k (our "win on time in rapidplay" case), re-ranking cannot recover it. In practice k is chosen to balance recall against re-ranking cost.

6 Conclusion

On the FIDE Laws of Chess, cross-encoder re-ranking fixed 10 of 30 test queries where the bi-encoder had buried the correct clause at ranks 4–8. It turned "Not found" answers into correct, grounded ones, at a cost of about 150 ms per query. The gains came mostly from rule variants in the appendices (Rapidplay, Blind chess, Chess960), where joint attention over the query and the chunk picks up qualifiers that a single pooled vector loses. Re-ranking is not free of errors: it occasionally demoted correct chunks, and it cannot help when the bi-encoder's Top 10 misses the answer entirely.